MyCodeSchool

Code review — colections-csharp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-16 — sha `ac84a76`

Scop: pasul 1 din `EXERCITII.pdf` — `IColectie<T>`, `ILista<T>`, `ListaInlantuita<T>` (commit-urile `2f62ee4` „Lista inlantuita start" și `ac84a76` „lista inlantuita").

Metodă: build pe .NET 8, apoi `Program.cs` al tău rulat ca atare și, pe o copie, 11 teste mici pe metodele listei: listă goală, un singur element, `Insert(0)`, `RemoveAt` la capete. Codul tău nu a fost atins.

Poarta de compilare: ✅ **0 erori,** ⚠️ **13 warning-uri** (`CS8618`, `CS8625`, `CS8603`), toate în `Node.cs` și `ListaInlantuita.cs`. TEORIE §1e le explică; „Gata când" cere 0.

Pe scurt:

- ✅ Cele trei probleme mari din `Lista.cs`-ul vechi sunt rezolvate: legătura e un câmp al nodului (`Tail.Next = ...`), `readonly` a dispărut, metodele primesc `T`, nu noduri. Lista goală există, `Count` și `Tail` sunt ținute la zi în `Add`.
- 🔴 **B1:** `foreach` crapă (`GetEnumerator` neimplementat) și `ToString` lipsește, deci `Pas1` nu poate scoate nicio linie din fișă.
- 🔴 **B2:** `RemoveAt(0)` scade `Count` de două ori; după el `AsVector` crapă.
- 🔴 **B3:** `Tail` nu e ținut la zi de constructor, `Clear` și `Insert(0)`: elemente pierdute sau `NullReferenceException` la următorul `Add`.
- 🔴 **B4:** trei metode cad pe lista goală sau pe lista cu un singur element.
- 🟡 `Head` folosit drept cursor în toate parcurgerile · numele din contract schimbate · nodul public · indexul greșit tace (M1–M4).

Ieșirea lui `Pas1`, rând cu rând

Rând din fișă	La tine	De ce
`[] Count=0`	`Colections.ListaInlantuita`1[System.String] Count=0`	B1 (`ToString`)
`[a, b, c, d, e] Count=5`	`... Count=5`	B1; `Count` corect

Rând din fișă	La tine	De ce
`True False`	`True False`	✅
`[b, D, f] Count=3`	`... Count=2`	B2
`2 -1`	`2 -1`	✅
`True False`	`True False`	✅
`b;D;f;`	`NotImplementedException`	B1, programul se oprește aici
ultimele 2 rânduri	nerulate	

🔴 Critice

B1 — `ListaInlantuita.cs:279–289` : `foreach` nu merge, `ToString` lipsește

Dovada: `dotnet run` → prima linie e `Colections.ListaInlantuita`1[System.String] Count=0`, iar la `Program.cs:22` ( `foreach` ) `Unhandled exception. System.NotImplementedException`.

Mecanism: `foreach`, `string.Join` și `Console.WriteLine(lista + ...)` nu știu nimic despre noduri. Primele două cer `GetEnumerator()`; al treilea cheamă `ToString()`, iar fără suprascriere `object.ToString()` întoarce numele tipului. Ai scris `//yield ????` exact în locul potrivit.

💡 TEORIE §6: bucla din `AsVector` (`:112–116`), scrisă cu `yield return` în loc de `array[index++] = ...`, devine enumerator; compilatorul face restul. Al doilea `GetEnumerator` (`:285–289`) e deja corect. 💡 `ToString`: din clipa în care `GetEnumerator` există, `this` e un `IEnumerable<T>` și `string.Join` îl primește direct.

B2 — `ListaInlantuita.cs:169–173` și `:198–200` : `RemoveAt(0)` scade `Count` de două ori

Dovada (copie): `[a, b, c, d, e]` → `RemoveAt(0)` → 4 noduri în lanț, dar `Count=3`; `AsVector` → `IndexOutOfRangeException` la `:114`. În `Pas1` se vede pe rândul 4: `Count=2` în loc de `Count=3`. La fel `RemoveAt(5)` pe 5 elemente: nu scoate nimic, dar `Count` devine 4.

Mecanism: ramura `index == 0` n-are `return`. După ce a scos capul, execuția continuă în bucla de dedesubt, ajunge la ultimul nod și intră în `:198–200`, linii scrise pentru alt caz, care scad `Count` încă o dată. Aceleași linii se ating și la `index == Count`, fiindcă garda de la `:167` e `>`, nu `>=`.

💡 Compară ramura `index == 0` din `Insert` (`:128–135`) cu cea din `RemoveAt` (`:169–173`): ce are una și nu are cealaltă? 💡 Pentru ce caz există `:198–200` ? Dacă garda de la `:167` ar respinge tot ce nu e în `[0, Count)` , mai poate ajunge execuția acolo?

B3 — `ListaInlantuita.cs:16` , `:103` , `:128–135` : `Tail` rămâne în urmă

Dovada (copie): listă nouă → `Insert(0, "a")` → `Add("b")` → `AsVector` = `[a, null]` , `Count=2` , `IndexOf("b") = -1` . „b" s-a legat de nodul gol din constructor, nu de „a". Același scenariu după `Clear()` → `NullReferenceException` la `:38` .

Mecanism: `Add` are încredere că `Tail` e ultimul nod din lanț. Două locuri îl lasă să mintă: constructorul pune în `Tail` un nod gol care nu e în listă (`:16`), iar `Insert(0, x)` pe lista goală schimbă `Head` fără să atingă `Tail` (`:128–135`). Orice metodă care schimbă capătul lanțului trebuie să schimbe și `Tail` ; `Remove` (`:62–64`) și `RemoveAt` (`:184–187`) o fac deja.

💡 Ce înseamnă „lista goală" pentru cele trei câmpuri, scris o singură dată, și care metode trebuie să o producă identic: constructorul, `Clear` , ultimul `Remove` ? 💡 `Insert(0, x)` pe lista goală e același caz ca `Add(x)` pe lista goală; codul îl ai la `:31–36` .

B4 — `ListaInlantuita.cs:47` , `:171` , `:178` : lista goală și lista cu un element

Dovada (copie): `Remove("x")` pe lista goală → `NullReferenceException` la `:47` · `RemoveAt(0)` pe lista goală → NRE la `:171` · `RemoveAt(0)` pe `[a]` → NRE la `:178` (după ce a scos capul, `Head` e `null` , iar bucla citește `Head.Next`). `Contains` și `IndexOf` sunt corecte pe lista goală.

Mecanism: TEORIE §1d, lista goală e o stare normală; `Head.Data` sau `Head.Next` pe `Head == null` e exact `NullReferenceException` . `Pas1` nu lovește niciunul dintre cazuri; `Pas6` le lovește pe toate.

💡 Fiecare metodă care începe cu `Head.` are nevoie de o întrebare înainte. `Add` o pune deja (`:31`).

🟡 Importante

- **M1** — `Head` e folosit drept cursor în toate cele 8 parcurgeri (`:55–75` , `:81–96` , `:108–119` , `:137–162` , `:175–201` , `:206–223` , `:234–249` , `:259–274`): `aux` salvează capul, `Head` umblă, la final `Head = aux` . Merge, dar între prima și ultima linie a fiecărei metode lista e stricată, și orice excepție pe drum (vezi B4) o lasă cu `Head` în mijloc. E oglinda greșelii din `Lista.cs` : acolo variabila locală încerca să fie câmp, aici câmpul e folosit drept variabilă locală. 💡 Cursorul e variabila locală (`Node<T> curent = Head`), câmpul nu se mișcă; `AsVector` și `IndexOf` sunt cele mai ușoare de început.
- **M2** — numele din contract sunt schimbate (`Add/Remove/Contains/Clear/AsVector/Insert/RemoveAt` în loc de `Adauga/Sterge/Contine/Goleste/CaTablou/Insereaza/StergeLa`), `Pas1()` nu există și `DemoLista` e chemată din `Main` . Consecința: `Pas2` … `Pas7` și `Sortat` din fișă nu se mai pot copia ca atare, le traduci la fiecare pas. Vorbim la ședință; oricum ar fi, aceleași nume peste tot.
- **M3** — `Node<T>` e public, cu câmpuri publice (`Node.cs:3–6`), deși `ILista<T>` nu-l cere nicăieri: poate fi deja `private class Nod` în listă (fișa, „Gata când"). Cele 13 warning-uri sunt toate despre `Next` , `Head` și `Tail` , care pot fi `null` dar sunt declarate ca și cum n-ar putea (`Node<T>?` , TEORIE §1e). Același `CS8618` pe care l-ai lăsat trei runde în `design-patterns` , la `AutoLicitator` .

- **M4** — indexul greșit tace: `Insert(index > Count)` și `RemoveAt` fac `return` (`:126`, `:167`), `this[i]` întoarce `default(T)` (`:231`, `:250`), iar indexul negativ nu e verificat deloc: `lista[-1]` dă `null`, `Insert(-1, x)` parcurge toată lista și nu face nimic. În `Pas6` devin `ArgumentOutOfRangeException`; până atunci `lista[7]` care „merge" pe o listă de 5 e un bug ascuns.

🟢 Cleanups

- **C1** — `ListaInlantuita.cs:2-3`: `System.Diagnostics.Metrics` și `System.Runtime.Intrinsics.X86`, `using`-uri puse de IDE la autocomplete, nefolosite.
- **C2** — `ListaInlantuita(T data)` (`:20-25`) și `Node()` (`Node.cs:8-11`) nu sunt în contract; al doilea există doar ca să creeze nodul gol din B3.
- **C3** — `//ICollection`, `//IList`, `//yield ????`, `//????`: notițe de lucru; întrebările merg în `RASPUNSURI.txt`.

Q&A

1. După `RemoveAt(0)` pe `[a, b, c, d, e]`, câte noduri sunt în lanț și cât e `Count`? De ce diferă?
2. `Tail` înseamnă „ultimul nod". Care metode din clasa ta pot schimba cine e ultimul nod? Enumeră-le și verifică la fiecare dacă îl actualizează.
3. De ce `Console.WriteLine(lista + " Count=" + lista.Count)` afișează numele tipului, nu elementele?

Răspunsurile în `RASPUNSURI.txt`, în repo.

Următorul pas: B1–B4, apoi `Pas1` identic cu fișa, 0 warning-uri, și abia apoi `Pas2`.